Stand & Deliver Demo Guide – Galaxium Travels

Step-by-step runbook for the IBM Bob Level 3 recording: prerequisites, exact Bob prompts, commands, and word-for-word speech.

Source documents: `IBM Bob L3 Solution Document.md` and `Stand and Deliver Use Case.md` **Use case:** "Stop customers cancelling flights they've already flown" **Length:** 8–10 minutes **Stage chosen:** Delegate (next: Tailor, later: Scale & Extend)

> Dollar figures are illustrative assumptions. Say so in the recording.

PART A – Prerequisites and project setup

A1. What you need installed

Tool	Needed for	Check command	Notes
Python **3.10+**	Backend	`python3 --version`	The code uses `BookingOut | ErrorResponse` type syntax, which needs 3.10 or newer. The README says 3.8+, which is out of date. Verified on 3.13.5.
Node.js 18+ and npm	Frontend (optional for this demo)	`node --version` and `npm --version`	Found on this machine: Node 25.6.0, npm 11.21.1.
`curl`	Calling the API during the demo	`curl --version`	Preinstalled on macOS.
`sqlite3` CLI	Finding a `completed` booking quickly	`sqlite3 --version`	Preinstalled on macOS. Optional. You can use `curl` instead (A5).
Git	Showing diffs	`git --version`	
IBM Bob (IDE)	The demo itself	Open the workspace	Use **Agent** mode for the live steps.

A2. Open the project in Bob

1. Open the folder `galaxium-travels` as the Bob workspace.
2. Confirm you are on branch `bob-learning-path-branch` (`git branch --show-current`).
3. Start from a clean state so the diffs on screen are only your demo changes. Many files show as modified in git status already. Commit or stash them first:

```
git add -A && git commit -m "Baseline before Bob demo"
```

If you cannot commit, at least note the current `git status` so you can tell your changes apart.

A3. Backend setup (required)

```
cd booking_system_backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Dependencies installed (from `requirements.txt`): fastapi, fastmcp, uvicorn, sqlalchemy, pydantic[email], python-dotenv, pytest, pytest-asyncio, pytest-cov, httpx.

Verify the baseline. This is the safety net you will mention in the demo:

```
pytest
```

Expected: `29 passed`. This was run during preparation. There are about 6 warnings, including `datetime.utcnow()` deprecation. Those are normal.

A4. Frontend setup (optional, only needed for step 6 if you want to show the UI)

```
cd booking_system_frontend
cp .env.example .env        # sets VITE_API_URL=http://localhost:8080
npm install
npm run dev                 # http://localhost:5173
```

The frontend is not required to run the demo. In step 6 Bob only reads `api.ts`.

A5. Start the backend and prepare the demo data

Open a **second terminal** for the server.

```
cd booking_system_backend
source .venv/bin/activate
python server.py
```

- API: http://localhost:8080
- Swagger UI: http://localhost:8080/docs
- MCP endpoint: http://localhost:8080/mcp
- Health check: `curl http://localhost:8080/` should return `{"status":"OK"}`

Important – seed data is random and resets on every start. `seed.py` deletes all data and recreates 10 users, 10 flights and 20 bookings, with random user, flight and status (`booked`, `cancelled`, `completed`). So you must find a `completed` booking **after the last server start**.

Find one (with the server running, from `booking_system_backend`):

```
sqlite3 booking.db "SELECT booking_id, user_id, flight_id, status FROM bookings WHERE status='completed' LIMIT 3;"
```

No `sqlite3`? Use curl across the 10 users:

```
for i in 1 2 3 4 5 6 7 8 9 10; do curl -s http://localhost:8080/bookings/$i; echo; done | grep -o '{[^}]*"completed"[^}]*}'
```

Write down:

Item	Your value
`BOOKING_ID` (a `completed` booking)	_____
`FLIGHT_ID` of that booking	_____
Seats on that flight before the cancel (`curl -s http://localhost:8080/flights`)	_____

> If the chance of no `completed` booking in 20 rows concerns you: with 3 equal statuses it is very unlikely (about 0.03%), but if it happens, restart the server and look again.

Do not restart the server between step 3 and step 7, or the booking ID will change.

A6. Pre-recording checklist

- [ ] `pytest` shows 29 passed
- [ ] Server runs on port 8080 and `curl localhost:8080/` returns OK
- [ ] You have a `BOOKING_ID`, its `FLIGHT_ID`, and the seat count written down
- [ ] Terminal 1 is in `booking_system_backend` with the venv active (for `pytest`)
- [ ] Terminal 2 runs the server
- [ ] Bob is in **Agent** mode, with the Galaxium workspace open
- [ ] `git status` is clean (or you know the baseline)
- [ ] You will say out loud: **Delegate**, and that the dollar figures are illustrative
- [ ] Closing statement ready with owners, dates, scope (see Part B, Section 5)
- [ ] Screen recording set up, font size large, notifications off

---

PART B – The demo, step by step

Timing overview:

Time	Section	Criterion
0:00–1:30	1. Pain and value	1
1:30–6:30	2. Live use case (8 steps)	2 (and 3, by narrating)

Time	Section	Criterion
6:30–7:30	3. Maturity curve	3
7:30–8:00	4. Recap (optional)	–
8:00–9:30	5. Call to action	4

Notation: **SAY** is what you speak. **DO** is what you click or type. **PROMPT** is pasted into Bob. **EXPECT** is what should appear. **IF IT DOESN'T** is a recovery line.

Section 1 – Pain and value (0:00–1:30) → Criterion 1

DO: Show the repo tree in Bob (`booking_system_backend` , `booking_system_frontend`) and keep `booking.py` ready.

SAY:

> "Hello, I'm [name]. Today I'm showing how IBM Bob helps **Galaxium Travels**, a mid-size interplanetary travel operator. They run their own booking platform, and I'm using that platform's code for this demo.
>
> Their problem is specific. They are about to let an **AI assistant cancel bookings** through their MCP tools. But today, a customer, or that AI agent, can cancel a booking that is already **completed**, and the system gives the seat back on a flight that has already flown. That is a revenue and trust problem.
>
> It is made worse by a people problem. Two of the three engineers who know this code are leaving this quarter.
>
> I've built a simple cost model. These are **illustrative assumptions**, not client data, and I'd replace them with Galaxium's own numbers. With 50,000 bookings a year and a 0.5% defect rate at about 400 dollars each, that's roughly **100,000 dollars** in refunds and seat errors. Add goodwill and rebooking, about **125,000**. Add slow onboarding from outdated docs, about **35,000**. That's around **260,000 dollars a year** from doing nothing.
>
> Bob's value here is simple: it lets the engineers who stay **find, prove and fix** this in minutes, and every change is a diff they review."

DO (optional): Open `booking.py` at line 67 and highlight the single `if booking.status == "cancelled"` check.

SAY (optional):

> "Here is the evidence: this function only blocks one status, `cancelled` . Nothing else."

Section 2 – Live use case (1:30–6:30) → Criterion 2

Step 1 – Understand the system (~0:40)

SAY:

> "First I ask Bob to explain the system, since the people who knew it are leaving."

PROMPT:

```
Explain this repo and how REST and MCP share business logic.
```

EXPECT: Bob explains that `server.py` is a thin layer over `services/*` (`flight.py`, `user.py`, `booking.py`). The REST endpoints and the six MCP tools both call the same functions.

SAY (after Bob answers):

> "Key point: REST and MCP share one service layer. So **one fix covers both** the website and any AI agent."

Step 2 – Review the cancellation rules (~0:30)

SAY:

> "Now I delegate a review to Bob. I'm not telling it where the bug is."

PROMPT:

```
Review cancel_booking in booking_system_backend/services/booking.py for business-rule
gaps. Do not change any code yet.
```

EXPECT: Bob says that only `cancelled` is rejected, so `completed` bookings can be cancelled, and that the seat is returned (`seats_available += 1`) for a flight that has already flown. It may also mention other items (non-atomic seat update, `datetime.utcnow()`).

SAY:

> "Bob found the gap: a completed booking can be cancelled, and the seat goes back to inventory. I'll focus on this one."

IF IT DOESN'T: Follow up with: `Does cancel_booking check the booking status other than 'cancelled'?`

Step 3 – Reproduce the bug (~0:50)

SAY:

> "Before I trust any finding, I prove it. I've started the API and picked a booking that is already completed."

DO (terminal 1, replace the placeholders):

```
# 1. Seats before
curl -s http://localhost:8080/flights | python3 -m json.tool | grep -B1 -A6 '"flight_id": FLIGHT_ID,'

# 2. Cancel a COMPLETED booking
curl -s -X POST http://localhost:8080/cancel/BOOKING_ID

# 3. Seats after
curl -s http://localhost:8080/flights | python3 -m json.tool | grep -B1 -A6 '"flight_id": FLIGHT_ID,'
```

If `grep` formatting is awkward, simply run `curl -s http://localhost:8080/flights` before and after and compare `seats_available` for your flight.

EXPECT: The cancel call returns a booking with `"status":"cancelled"`, and `seats_available` for that flight has gone up by 1.

SAY:

> "Confirmed. The API accepted the cancellation of a completed booking, and the seat count went up. This is a real defect, not a guess."

IF IT DOESN'T: You probably restarted the server and the IDs changed. Re-run the lookup in A5.

> After this step, that booking is `cancelled`. For step 7 you need the **same call to fail**, so pick a **second** `completed` booking now and note it as `BOOKING_ID_2` (you also need its `FLIGHT_ID_2` and seat count). If there is only one, restart the server before step 7 and use a new one.

Step 4 – Tests first (~0:50)

SAY:

> "Now tests first. I ask Bob to write a test that fails, so the bug is captured in code."

PROMPT:

```
Add a failing test to booking_system_backend/tests/test_services.py for cancelling a completed booking. It should create a user, a flight and a booking with status "completed", call booking.cancel_booking, and assert the result is an ErrorResponse with error_code "INVALID_STATUS" and that the flight's seats_available is unchanged. Follow the style of the existing tests. Do not change booking.py.
```

DO: Review the diff, accept it, then in terminal 1:

```
pytest tests/test_services.py -k completed
```

EXPECT: The new test **fails**. (It fails because the function currently returns a cancelled `BookingOut`, not an `ErrorResponse`.)

SAY:

> "The new test fails. That proves the bug in a repeatable way, and it protects the fix later."

Step 5 – Minimal fix (~0:50)

SAY:

> "Now I ask for the smallest possible fix and tell Bob to reuse the error pattern the codebase already has."

PROMPT:

```
Fix cancel_booking in services/booking.py minimally so the new test passes. Reuse the existing ErrorResponse pattern. Keep the existing ALREADY_CANCELLED behaviour for cancelled bookings, and return error_code "INVALID_STATUS" for any other status that is not "booked". Do not touch anything else.
```

EXPECT: A small diff in `booking.py`, roughly:

```
if booking.status == "cancelled":
    ...ALREADY_CANCELLED...

if booking.status != "booked":
    return ErrorResponse(
        error="Booking cannot be cancelled",
        error_code="INVALID_STATUS",
        details=f"Booking {booking_id} has status '{booking.status}' and can only be
cancelled when status is 'booked'.",
    )
```

placed **before** the "Restore seat" block.

SAY:

> "This is the review moment. I read the diff, it's a few lines, it follows the existing pattern, and I accept it. The engineer stays in control."

DO: Accept the diff.

Step 6 – The unexpected obstacle (~1:00)

SAY:

> "Here Bob raised something I didn't expect. A bug like this would normally return a proper HTTP error, like 409. So I ask Bob whether we should do that. But I also want to know what it does to the UI before we touch anything."

PROMPT:

```
Should this return a 4xx HTTP status instead of 200? Check how the frontend handles
errors in booking_system_frontend/src/services/api.ts before you recommend anything. Do
not change code yet.
```

EXPECT: Bob inspects `api.ts` :

- `isErrorResponse` (line 109) decides what is an error from the response body.
- The response interceptor (lines 20–35) rejects with `error.response.data`.
- A bare FastAPI `HTTPException` returns `{ "detail": ... }`, which does **not** match the `ErrorResponse` shape. The UI's error handling would break.
- Recommendation: keep the existing `ErrorResponse` shape for now. Treat a proper 4xx as a follow-up that changes backend and frontend together.

SAY:

> "This is the obstacle. The obvious fix, returning a 4xx, would have silently broken the UI. Bob checked the impact on another part of the system **before** changing anything, and recommended keeping the current `ErrorResponse` shape. The 4xx change goes on the follow-up list. That's the difference between an assistant that guesses and one that checks."

IF IT DOESN'T: Prompt: `What would the frontend receive if the backend raised HTTPException(409)? Quote the relevant lines of api.ts.`

Step 7 – Verify (~0:40)

SAY:

> "Now I verify, with the tests and with the same call that failed before."

DO (terminal 1):

```
pytest
```

EXPECT: All tests pass (30 if one test was added).

Restart the server so it loads the fix, then use a **completed** booking:

DO:

1. In terminal 2 press `Ctrl+C`, then run `python server.py` again.
2. **The seed is random again, so repeat the lookup from A5** and note a new `BOOKING_ID` and its flight's seat count.
3. ```
   curl -s -X POST http://localhost:8080/cancel/BOOKING_ID
   curl -s http://localhost:8080/flights
   ```

> Why restart: `python server.py` does not auto-reload, so the running server still has the old code. If you prefer not to restart and look up a new ID, you can run the server with reload once during preparation: `uvicorn server:app --port 8080 --reload`. Note the seed would still run on every reload.

EXPECT: The response is an `ErrorResponse` containing `"error_code":"INVALID_STATUS"`. Seat counts are unchanged.

SAY:

> "Tests pass, and the same kind of call now returns `INVALID_STATUS`. The seat count didn't move. The bug is closed for the website and for any AI agent, because they share this function."
   ```

Step 8 – Docs (~0:40)

SAY:

> "Last, the documentation. Outdated docs are part of why onboarding is slow."

PROMPT:

```
Update booking_system_backend/README.md so the endpoints match what server.py really
serves. The README says /api/flights, /api/book and so on, but the code serves /flights,
/book, /bookings/{user_id}, /cancel/{booking_id}, /register and /user. Also remove the
stray word "doc" on its own line, and mention the INVALID_STATUS error for cancelling.
```

EXPECT: `/api/*` paths become `/flights`, `/book`, etc., in the endpoint table and the `curl` examples. The stray `doc` line is removed. `INVALID_STATUS` is documented.

SAY:

> "Bob compared the README with the real code and fixed the drift. The next engineer who joins will read documentation that is true."

DO: Show the diff, accept it, then run `git diff --stat` to show the small change set.

SAY:

> "Everything I did is a small diff I reviewed: one test, a few lines of fix, and the docs."

Section 3 – AI Maturity Curve (6:30–7:30) → Criterion 3

DO: Optionally show a slide with four stages: Assist, Delegate, Tailor, Scale & Extend.

SAY:

> "Where does this client sit on the AI Maturity Curve? I've positioned them at **Delegate**.
>
> At Assist, AI answers questions. At **Delegate**, you give AI a whole task, with the engineers approving. That's what you just saw: I gave Bob a task chain of explain, review, reproduce, write a failing test, make the minimal fix, check the frontend impact, verify, and update the docs. I reviewed every step.
>
> Delegate is the right stage for Galaxium because their pain is **capacity and knowledge loss**. They are short of people, not ideas. And the repo already has what safe delegation needs: a service layer, tests, typed schemas, and error codes.

> **Next is Tailor.** We would create custom Bob modes and rules that encode Galaxium's own standards, for example: 'every business-rule failure returns an ErrorResponse with an error_code', and 'no `utcnow()`'. Every engineer then gets the same quality.
>
> **Later is Scale and Extend.** The MCP tools in this repo become the safe channel for customer-facing AI agents, and we apply the same approach to Galaxium's other platforms."

Section 4 – Recap and "what's next" (7:30–8:00, optional)

SAY:

> "To recap what I did: I explained the system, found and reproduced a real defect, captured it in a failing test, fixed it with a small diff, checked the frontend impact before touching the API contract, verified, and refreshed the docs.
>
> Two follow-ups are already on the list because Bob surfaced them: returning proper 4xx statuses with a coordinated frontend change, and making the seat update atomic so two people can't take the last seat at the same time."

Section 5 – Call to action (8:00–9:30) → Criterion 4

DO: Show a closing slide with: Offer, Scope, Owners, Dates, Success criteria.

SAY:

> "So here is my proposal. I'd like to run a **2-week Proof of Experience** on your booking service layer. We run this same workflow on your real code, targeting your top booking and cancellation defects.
>
> **Scope:** one booking repository: architecture explanation, a review of booking and cancellation logic, tests plus fixes for the top defects, and a README refresh.
>
> **Success criteria:** at least three verified defects fixed, more tests on the booking module, and documentation that matches the code.
>
> **Owners:** on your side, the CTO as sponsor and your Lead Engineer as day-to-day owner. On IBM's side, I lead the engagement and a technical specialist sets up Bob.
>
> **Timing:** a pre-work call within one week, kickoff at the start of your next sprint, and a readout on day 10 to 14. What we need from you is read access to one repo, a non-production environment, and two engineers for about three hours a week.
>
> This directly addresses the cancellation defect, prepares you for your AI assistant, and reduces the knowledge-loss risk.

> **Can we lock in a kickoff date today? Which engineer should own this on your side, and does the sprint starting [date] work?**"

If the client hesitates (fallback):

> "If a two-week PoX is too much right now, a smaller step is a **90-minute hands-on workshop on your own code** this month. We decide on the PoX afterwards."

Final line:

> "I'll confirm owners, dates and scope by email today. Thank you."

PART C – Quick reference

C1. All prompts, in order

#	Prompt
1	`Explain this repo and how REST and MCP share business logic.`
2	`Review cancel_booking in booking_system_backend/services/booking.py for business-rule gaps. Do not change any code yet.`
4	`Add a failing test to booking_system_backend/tests/test_services.py for cancelling a completed booking. It should create a user, a flight and a booking with status "completed", call booking.cancel_booking, and assert the result is an ErrorResponse with error_code "INVALID_STATUS" and that the flight's seats_available is unchanged. Follow the style of the existing tests. Do not change booking.py.`
5	`Fix cancel_booking in services/booking.py minimally so the new test passes. Reuse the existing ErrorResponse pattern. Keep the existing ALREADY_CANCELLED behaviour for cancelled bookings, and return error_code "INVALID_STATUS" for any other status that is not "booked". Do not touch anything else.`
6	`Should this return a 4xx HTTP status instead of 200? Check how the frontend handles errors in booking_system_frontend/src/services/api.ts before you recommend anything. Do not change code yet.`
8	`Update booking_system_backend/README.md so the endpoints match what server.py really serves. The README says /api/flights, /api/book and so on, but the code serves /flights, /book, /bookings/{user_id}, /cancel/{booking_id}, /register and /user. Also remove the stray word "doc" on its own line, and mention the INVALID_STATUS error for cancelling.`

Steps 3 and 7 are commands, not prompts.

C2. Commands cheat sheet

Purpose	Command (from `booking_system_backend`, venv active)
Run tests	`pytest`
Run new test only	`pytest tests/test_services.py -k completed`
Start server	`python server.py`
Find completed bookings	`sqlite3 booking.db "SELECT booking_id, user_id, flight_id, status FROM bookings WHERE status='completed' LIMIT 3;"`
Cancel	`curl -s -X POST http://localhost:8080/cancel/BOOKING_ID`
Flights and seats	`curl -s http://localhost:8080/flights`
Show change set	`git diff --stat`
Undo demo changes to rehearse again	`git checkout -- booking_system_backend/services/booking.py booking_system_backend/tests/test_services.py booking_system_backend/README.md` (only safe if you committed the baseline in A2)

C3. Troubleshooting

Problem	Cause	Fix
`TypeError: unsupported operand type(s) for |` on start	Python older than 3.10	Install Python 3.10+ and recreate the venv
`Address already in use` on port 8080	An old server is running	`lsof -i :8080`, then `kill <pid>`
Booking ID not found	Server restarted, seed re-ran	Repeat the lookup in A5
No `completed` booking found	Random seed	Restart the server and look again
Cancel call still succeeds after the fix	Server still running old code	Restart `python server.py`
New test passes before the fix	Test does not use a `completed` booking	Ask Bob to set `status="completed"` and re-run
`ModuleNotFoundError` when running `pytest`	Venv not active or wrong folder	`source .venv/bin/activate` and `cd booking_system_backend`
Frontend can't reach API	Missing `.env` or backend down	`cp .env.example .env` and check `curl localhost:8080/`
Port 5173 busy	Another Vite instance	Stop it, or Vite will pick the next port

C4. Checklist mapping

Criterion	Where covered
1. Pain and value	Section 1 – code evidence, cost model, value line

Criterion	Where covered
2. Use case with obstacle	Section 2 – eight steps, obstacle in step 6
3. AI Maturity Curve	Section 3 – Delegate chosen with reasoning, then Tailor, then Scale & Extend; hands-on steps narrated throughout
4. Call to action	Section 5 – 2-week PoX, owners, dates, success criteria, fallback, commitment question

C5. Notes

- Baseline `pytest` was run while preparing this guide: **29 passed** on Python 3.13.5. The frontend build was not run.
- The fix, test and README changes in the demo are **not applied** to the repo. Bob produces them live.
- Exact Bob wording will vary between runs. Use the "EXPECT" lines to judge whether it is on track, and the "IF IT DOESN'T" prompts to steer it.
- The root `README.md` references `start.bat`, which does not exist. Only `start.sh` is present. `./start.sh` on macOS/Linux starts both backend and frontend if you prefer one command.
- `./start.sh` creates `.venv` and installs dependencies itself, but it starts the backend in the background, so use the manual steps in A3–A5 for the demo, where you need a separate terminal and a visible server.